

MICROSERVICES ARCHITECTURE

PRODUCT SERVICE

Product Catalog • Inventory • Categories • Search • Reviews

Service Port 8082	Database MySQL	Spring Boot 3.2.x / Java 21	Pattern Domain-Driven
----------------------	-------------------	--------------------------------	--------------------------

Enterprise-Grade • Industry Standard • Production Ready

Includes: Registration Flow • All APIs • DTOs • Entities • Security • Cache • Kafka • Batch • Docker

1. Service Overview

The Product Service owns the entire product catalog domain. It manages products, categories, inventory stock levels, product images, and customer reviews. It exposes both public read APIs (browsing, searching) and protected write APIs (admin product management).

Order Service calls Product Service via Feign Client to validate product availability and deduct inventory during order placement. Inventory updates are published as Kafka events for other services to consume.

Attribute	Value
Service Name	product-service
Base Package	com.ecommerce.productservice
Server Port	8082
Context Path	/api/v1
Database	MySQL 8.0 — dedicated product_db
Cache	Redis — product cache, category tree, search results
Messaging	Kafka Producer/Consumer — inventory events
Search	Full-text search via MySQL FULLTEXT index + Specification API
API Docs	http://localhost:8082/swagger-ui.html

2. Product Creation Flow

2.1 Create Product Sequence

Step	Action
1 — Admin Request	POST /api/v1/admin/products with CreateProductRequest — requires ROLE_ADMIN
2 — Validate Input	@Valid — validates required fields, price > 0, stock >= 0, category exists
3 — Category Check	Verify categoryId exists in categories table — throw ResourceNotFoundException if not
4 — SKU Uniqueness	Check sku uniqueness — throw ProductAlreadyExistsException (409) if duplicate SKU
5 — Persist Product	Save Product entity with status=DRAFT by default
6 — Save Images	If images provided in request, save to product_images table (URLs)
7 — Create Inventory	Create ProductInventory record: quantity, lowStockThreshold, warehouseLocation
8 — Publish Kafka Event	ProductCreatedEvent → topic: product.created
9 — Evict Cache	@CacheEvict on product-list and category-products caches

10 — Return Response	201 CREATED with full ProductDetailResponse
----------------------	---

2.2 CreateProductRequest DTO

Field	Type / Constraint	Validation
name	String, @NotBlank	@Size(min=3, max=200)
description	String, @NotBlank	@Size(min=10, max=5000)
shortDescription	String, optional	@Size(max=500) — for listing cards
sku	String, @NotBlank	@Pattern(regex="^[A-Z0-9-]{5,30}\$") — uppercase alphanumeric
price	BigDecimal, @NotNull	@DecimalMin("0.01") — must be positive
compareAtPrice	BigDecimal, optional	Original price for showing discounts
costPrice	BigDecimal, optional	Internal cost price (admin only)
categoryId	UUID, @NotNull	Must reference existing category
brandName	String, optional	@Size(max=100)
tags	List<String>, optional	Product tags for search
weight	Double, optional	In kilograms — for shipping calculation
initialStock	int, @Min(0)	Initial inventory quantity
lowStockThreshold	int, @Min(1)	Alert threshold — default 10
imageUrls	List<String>, optional	Product image URLs — first is primary
attributes	Map<String,String>	Dynamic attributes: { color: Red, size: XL }

3. Product APIs — /api/v1/products (Public)

Method	Endpoint	Auth	Description
GET	/products	Public	Paginated product listing — ?page=0&size=20&sort=price,asc&categoryId=&minPrice=&maxPrice=&inStock=
GET	/products/{id}	Public	Get product by UUID — increments view count via async event
GET	/products/sku/{sku}	Public	Get product by SKU code
GET	/products/search	Public	Full-text search: ?q=laptop&categoryId=&minPrice=&maxPrice=&brand=&sort=relevance
GET	/products/{id}/related	Public	Get related products (same category, similar tags) — paginated

GET	/products/{id}/reviews	JWT Bearer	Get product reviews — paginated, sorted by rating/date
POST	/products/{id}/reviews	JWT Bearer	Submit review — body: { rating(1-5), title, comment }
GET	/products/featured	Public	Get featured products — cached 1 hour
GET	/products/new-arrivals	Public	Products created in last 30 days — paginated
GET	/products/best-sellers	Public	Top-selling products from Order Service data — cached

4. Admin Product APIs — /api/v1/admin/products

Method	Endpoint	Auth	Description
GET	/admin/products	ROLE_ADMIN	All products including DRAFT status, paginated with all filters
POST	/admin/products	ROLE_ADMIN	Create product — full CreateProductRequest
GET	/admin/products/{id}	ROLE_ADMIN	Get product with internal fields (costPrice, profit margin)
PUT	/admin/products/{id}	ROLE_ADMIN	Full product update
PATCH	/admin/products/{id}	ROLE_ADMIN	Partial update — only provided fields
DELETE	/admin/products/{id}	ROLE_ADMIN	Soft delete — status=DELETED, not removed from DB
PATCH	/admin/products/{id}/status	ROLE_ADMIN	Publish/Unpublish: DRAFT → ACTIVE → INACTIVE → DELETED
POST	/admin/products/{id}/images	ROLE_ADMIN	Upload product images (multipart) — max 10 images
DELETE	/admin/products/{id}/images/{imageId}	ROLE_ADMIN	Remove product image
PATCH	/admin/products/{id}/inventory	ROLE_ADMIN	Update stock: { quantity, operation: ADD/SUBTRACT/SET, reason }
GET	/admin/products/low-stock	ROLE_ADMIN	Products below lowStockThreshold — paginated alert list
POST	/admin/products/bulk-import	ROLE_ADMIN	Bulk import via CSV — Spring Batch job

GET	/admin/products/statistics	ROLE_ADMIN	Stats: total, active, draft, out-of-stock, low-stock counts
-----	----------------------------	------------	---

5. Category APIs — /api/v1/categories

Method	Endpoint	Auth	Description
GET	/categories	Public	Full category tree (hierarchical — parent + children) — cached 1 hour
GET	/categories/{id}	Public	Get category by ID with subcategories
GET	/categories/{id}/products	Public	Paginated products in a category (includes subcategories)
POST	/admin/categories	ROLE_ADMIN	Create category — { name, slug, parentId(optional), imageUrl, description }
PUT	/admin/categories/{id}	ROLE_ADMIN	Update category
DELETE	/admin/categories/{id}	ROLE_ADMIN	Delete if empty — throw CategoryNotEmptyException if has products
PATCH	/admin/categories/{id}/reorder	ROLE_ADMIN	Update display order for category sorting

6. Internal Service APIs — /api/v1/internal

Called by Order Service via Feign Client for inventory validation and deduction.

Method	Endpoint	Caller	Description
GET	/internal/products/{id}/availability	Order Service	Check if product is available: { available: true, stock: 50, price: 299.99 }
POST	/internal/products/validate-order-items	Order Service	Batch validate: List<{productId, quantity}> → List<ValidationResult>
POST	/internal/products/reserve-stock	Order Service	Reserve inventory for pending order — idempotent with orderId
POST	/internal/products/confirm-stock	Order Service	Confirm (deduct) reserved stock after payment
POST	/internal/products/release-stock	Order Service	Release reserved stock on order cancellation

GET	/internal/products/{id}/price	Order Service	Get current price for order value calculation
-----	-------------------------------	---------------	---

7. JPA Entities

Entity / Table	Key Columns
Product / products	id(UUID PK), name, description, short_description, sku(UNIQUE), price(DECIMAL 10,2), compare_at_price, cost_price, category_id(FK), brand_name, status(ENUM:DRAFT/ACTIVE/INACTIVE/DELETED), view_count, sold_count, average_rating, review_count, weight, tags(JSON), attributes(JSON), created_at, updated_at, version
Category / categories	id(UUID PK), name, slug(UNIQUE), description, parent_id(FK self-ref nullable), image_url, display_order, is_active, created_at
ProductImage / product_images	id(UUID PK), product_id(FK), image_url, alt_text, display_order, is_primary
ProductInventory / inventories	id(UUID PK), product_id(FK UNIQUE), quantity(INT), reserved_quantity, low_stock_threshold, warehouse_location, last_updated
ProductReview / product_reviews	id(UUID PK), product_id(FK), user_id, rating(1-5), title, comment, is_verified_purchase, is_approved, helpful_count, created_at
StockMovement / stock_movements	id(UUID PK), product_id(FK), movement_type(ENUM:PURCHASE/SALE/RETURN/ADJUSTMENT), quantity, reference_id, reason, created_at

8. Kafka Events

Topic	Event	Payload
product.created	ProductCreatedEvent	productId, name, sku, categoryId, price, timestamp
product.updated	ProductUpdatedEvent	productId, updatedFields, timestamp
product.deleted	ProductDeletedEvent	productId, sku, timestamp
product.stock-updated	StockUpdatedEvent	productId, oldQty, newQty, operation, orderId, timestamp
product.low-stock-alert	LowStockAlertEvent	productId, name, currentQty, threshold, timestamp
product.out-of-stock	OutOfStockEvent	productId, name, timestamp
product.back-in-stock	BackInStockEvent	productId, name, newQty, timestamp
product.price-changed	PriceChangedEvent	productId, oldPrice, newPrice, timestamp

9. Search & Filter Query Params

Parameter	Type / Description
q	String — full-text search query (MATCH AGAINST in MySQL)
categoryId	UUID — filter by category (includes subcategories recursively)
minPrice	BigDecimal — minimum price filter
maxPrice	BigDecimal — maximum price filter
brand	String — filter by brand name (ILIKE)
inStock	boolean — true = only in-stock products
minRating	double — minimum average rating (1.0 - 5.0)
tags	List<String> — products containing any of these tags
sort	relevance price,asc price,desc rating,desc newest best-selling
page	int, default=0
size	int, default=20, max=100

10. Redis Cache

Cache Key	TTL	Evict On
<code>products::id</code>	15 min	Product update, delete, price/stock change
<code>products::sku::sku</code>	15 min	Product update or delete
<code>products::featured</code>	1 hour	Any product status change
<code>products::new-arrivals</code>	30 min	New product created
<code>categories::tree</code>	1 hour	Any category create/update/delete
<code>categories::id::products</code>	10 min	Product added/removed from category
<code>search::queryHash</code>	5 min	Any product update (broad eviction)
<code>inventory::productId</code>	2 min	Any stock movement

11. Exception Handling

Exception	HTTP Status	When
<code>ProductNotFoundException</code>	404	Product not found by ID or SKU
<code>CategoryNotFoundException</code>	404	Category not found

<code>ProductAlreadyExistsException</code>	409	Duplicate SKU
<code>CategoryNotEmptyException</code>	400	Delete category that has products
<code>InsufficientStockException</code>	409	Order tries to reserve more than available
<code>InvalidPriceException</code>	400	Price ≤ 0 or <code>compareAtPrice < price</code>
<code>ImageLimitException</code>	400	More than 10 images per product
<code>ReviewAlreadyExistsException</code>	409	User tries to review same product twice

12. Dependencies

pom.xml — Product Service Additional Dependencies

```

<!-- All dependencies from User Service PLUS: -->

<!-- MySQL Driver (instead of PostgreSQL) -->
<dependency><groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId><scope>runtime</scope></dependency>

<!-- Flyway MySQL -->
<dependency><groupId>org.flywaydb</groupId>
  <artifactId>flyway-mysql</artifactId></dependency>

<!-- OpenFeign — called by Order Service -->
<dependency><groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId></dependency>

<!-- AWS S3 for production image storage -->
<dependency><groupId>software.amazon.awssdk</groupId>
  <artifactId>s3</artifactId><version>2.25.x</version></dependency>

<!-- Elasticsearch (optional — for advanced search) -->
<dependency><groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-elasticsearch</artifactId></dependency>

```